

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №5

Выполнил:

Майстренко Анастасия Николаевна

K3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подключить и настроить брокер сообщений RabbitMQ и реализовать межсервисное взаимодействие микросервисов сервиса аренды недвижимости (JP2) посредством очередей сообщений.

Ход работы

1. Подключение и настройка RabbitMQ

RabbitMQ поднимается через docker-compose (образ rabbitmq:3-management): порт 5672 — протокол AMQP, порт 15672 — веб-панель управления (guest/guest).

```
docker compose up -d rabbitmq
```

В сервисы добавлена библиотека amqp-lib. Логика работы с брокером вынесена в общий модуль shared/broker.ts с функциями connectBroker (подключение с повторными попытками), publishEvent (публикация события) и consumeEvents (подписка). При старте каждый сервис объявляет общий topic-exchange rental.events (durable).

2. Схема обмена событиями

Использован topic-exchange: издатель публикует событие с ключом маршрутизации (routing key), а каждый сервис-подписчик создаёт собственную durable-очередь и привязывает её к нужным ключам. Это позволяет одному событию независимо обрабатываться несколькими подписчиками.

Таблица 1 — Событийное взаимодействие

Событие (routing key)	Издатель	Подписчики и реакция
booking.confirmed	Booking	Catalog → объект в статус rented; Messaging → автодиалог арендатора с владельцем
booking.completed	Booking	Catalog → объект в статус available
booking.cancelled	Booking	Catalog → объект в статус available

Очереди подписчиков: catalog.booking-events (ключи booking.confirmed, booking.active, booking.completed, booking.cancelled) и messaging.booking-events (ключ booking.confirmed).

3. Издатель событий (Booking Service)

При смене статуса сделки (эндпоинт PATCH /bookings/{id}/status) Booking Service вместо синхронного REST-вызова публикует доменное событие в RabbitMQ:

```
publishEvent(`booking.${status}`, {
```

```
    bookingId, propertyId, tenantId, ownerId
  });
```

Это разрывает прямую зависимость между сервисами: Booking не знает, кто и как обрабатывает событие, и не блокируется на время обработки.

4. Подписчики событий

Catalog Service (shared/broker → consumeEvents в services/catalog/events.ts) слушает события бронирования и асинхронно обновляет статус объекта: при подтверждении/активации — rented, при завершении/отмене — available.

Messaging Service слушает booking.confirmed и автоматически открывает диалог между арендатором и владельцем объекта (если он ещё не существует), используя ownerId из полезной нагрузки события.

5. Надёжность доставки

Для устойчивости обмена сообщениями применены:

- durable exchange и durable очереди — переживают перезапуск брокера;
- persistent-сообщения — сохраняются на диск;
- подтверждение обработки ask; для «битых» сообщений — nack без повторной постановки;
- переподключение к брокеру с повторными попытками (сервис стартует даже раньше RabbitMQ).

6. Проверка работоспособности

Все сервисы и RabbitMQ запущены, выполнен сквозной сценарий. Результаты — в таблице 2.

Таблица 2 — Результаты проверки

Проверка	Результат
Все сервисы подключились к RabbitMQ (exchange rental.events)	ok
Подписчики создали очереди и привязали routing keys	ok
Подтверждение брони → Booking опубликовал booking.confirmed	опубликовано
Событие получено Catalog И Messaging (один→несколько)	оба получили
Catalog по событию: объект available → rented	rented
Messaging по событию: автоматически открыт диалог	диалог создан
Завершение брони → booking.completed → объект available	available
Очереди после обработки (management API)	0 непрочитанных

Логи наглядно показывают цепочку: «опубликовано событие booking.confirmed» → «получено событие» в обеих очередях → обновление статуса объекта и открытие диалога.

Вывод

В ходе работы подключён и настроен брокер сообщений RabbitMQ, а межсервисное взаимодействие микросервисов сервиса аренды недвижимости переведено на асинхронную модель через очереди сообщений. Booking Service публикует доменные события об изменении статуса сделки в topic-exchange, а Catalog и Messaging независимо обрабатывают их в собственных очередях, обновляя статус объекта и открывая диалог.

Такой подход обеспечивает слабую связанность сервисов, устойчивость к временной недоступности подписчиков и возможность добавлять новых потребителей событий без изменения издателя. Работоспособность подтверждена сквозным сценарием: одно событие booking.confirmed успешно обработано сразу двумя сервисами.